

# Data Visualization

BIOL 3401 Mount Royal University

Topic 5, Winter 2026

## Contents

|                                                |           |
|------------------------------------------------|-----------|
| Data Visualization . . . . .                   | 2         |
| <b>1 - Set up your R workspace for the day</b> | <b>2</b>  |
| Step 1a: Set your working directory . . . . .  | 2         |
| Step 1b: Load packages . . . . .               | 2         |
| Step 1c: Load data . . . . .                   | 2         |
| <b>2 - Construct a basic scatterplot</b>       | <b>3</b>  |
| Step 2a: ggplot2 basics . . . . .              | 3         |
| Step 2b: Plot a scatterplot . . . . .          | 4         |
| Step 2c: Rescale variables . . . . .           | 5         |
| Step 2d: Adding a trend line . . . . .         | 6         |
| Step 2e: Rename the axis labels . . . . .      | 7         |
| Step 2f: Colour points by group . . . . .      | 7         |
| <b>3 - Construct a boxplot</b>                 | <b>9</b>  |
| Step 3a: Basic boxplot . . . . .               | 9         |
| Step 3b: Facetting . . . . .                   | 10        |
| <b>Step 4 - Construct a line graph</b>         | <b>12</b> |
| Step 4a: Basic line graph . . . . .            | 12        |
| Step 4b: Customize your colours . . . . .      | 12        |
| <b>Step 5 - Construct a bar chart</b>          | <b>14</b> |
| Step 5a: Basic bar chart . . . . .             | 14        |
| Step 5b: Factor the treatments . . . . .       | 15        |
| Step 5c: Fill the bars by a variable . . . . . | 16        |

## Data Visualization

`ggplot2` (referred to simply as ‘`ggplot`’) is a `tidyverse` package that is basically the gold standard for plotting data when using R. The basic anatomy of a `ggplot` consists of the following elements:

| Syntax                | Description                              |
|-----------------------|------------------------------------------|
| <code>ggplot()</code> | Creates a <code>ggplot</code>            |
| <code>aes()</code>    | Constructs aesthetic mappings            |
| <code>+</code>        | Adds a component to your plot            |
| <code>geom_</code>    | Geometric layer (e.g., points, lines...) |

With these basic elements, you can create a high quality figure, and there are many aesthetic components you can add/change as well. The workflow for making a `ggplot` is typically an iterative process where you gradually refine your work towards your finished project.

## 1 - Set up your R workspace for the day

### Step 1a: Set your working directory

Make sure we’re setting our working directory!

```
# Set your working directory, then remove 'eval = F'!  
setwd()
```

### Step 1b: Load packages

```
# Load tidyverse (ggplot2 is a part of the tidyverse, so you already have it  
# installed)  
library(tidyverse)
```

### Step 1c: Load data

We are using four datasets today. All four are `.csv` files. We are going to use them to create four different plots, visualizing the patterns and adding new `ggplot2` elements as we go. The datasets are as follows:

- **cell\_metabolism.csv:** Contains 150 observations from three human cell types. We are going to use the data to visualize the relationship between cell size and metabolic rate.
  - `cell_type`: The type of cell (epithelial, neuron, or muscle)
  - `length_um`: The length of the cell in micrometers
  - `metabolic_rate_pmol_min`: The metabolic rate of the cell in picomoles per minute
- **cortisol\_stress.csv:** Contains 240 observations from an experiment where human subjects experienced stressor “treatments”, and then blood samples were taken to measure their circulating blood cortisol (stress hormone). We are going to use the data to visualize the relationship between human exposure to stressors and stress levels.
  - `treatment`: The stressor the subject was exposed to (Control, Heat stress, or Diet restriction)
  - `sex`: M (male) or F (female) subjects

- cortisol\_ng\_ml: The concentration of circulating blood cortisol after exposure to the stressor in nanograms per millileter
- **exercise\_recovery.csv**: Contains 63 observations of human heart rates following exercise. We are going to use the data to visualize cardiovascular recovery over time.
  - time\_min: Time since stopping exercise in minutes
  - heart\_rate\_bpm: Heart rate of the subject in beats per minute
  - fitness\_level: Relative fitness level of the subject (Low, Moderate, or High)
- **fish\_stocking.csv**: Contains 8 observations of fish survival under different hatchery stocking densities (fish per m<sup>2</sup>). We are going to use the data to compare survival by stocking density.
  - density: The density treatment where the fish were stocked (Low, Medium, High, or Very High)
  - environment: The setting where the fish was stocked (Artificial tank or Natural pond)
  - n\_stocked: Number of fish stocked in the treatment
  - n\_survived: The number of fish that survived out of those that were stocked
  - survival\_proportion: The proportion of surviving fish out of those that were stocked

Let's load the datasets now! Finish the `read_csv()` functions.

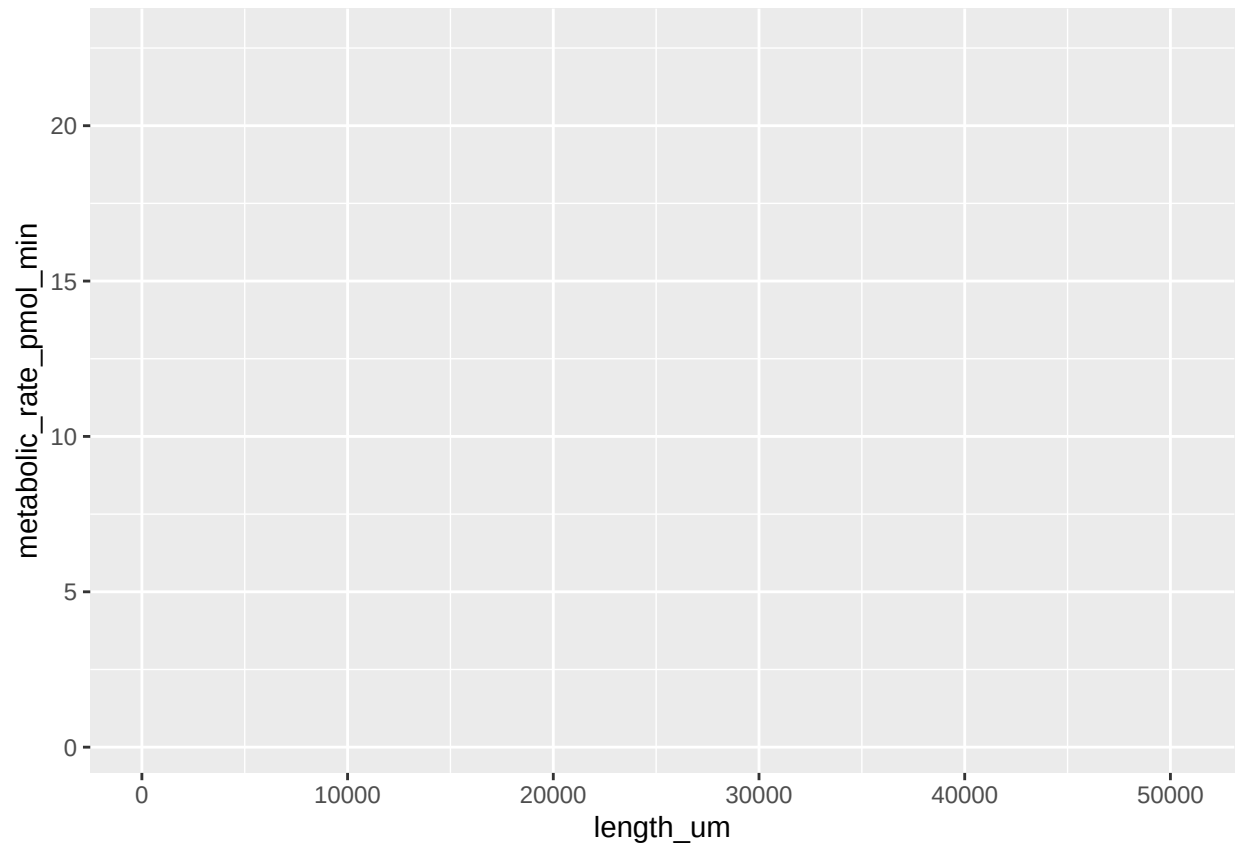
## 2 - Construct a basic scatterplot

We are going to start with `cell_data`. We are going to construct a scatterplot to visualize the relationship between cell size and metabolic rate.

### Step 2a: ggplot2 basics

The first layer to specify when making figures is the global axes. These axes will apply to all layers that follow this first one. This is typically where you want to specify your main axes (in the aesthetic) and the dataset.

```
ggplot(cell_data, aes(x = length_um, y = metabolic_rate_pmol_min))
```

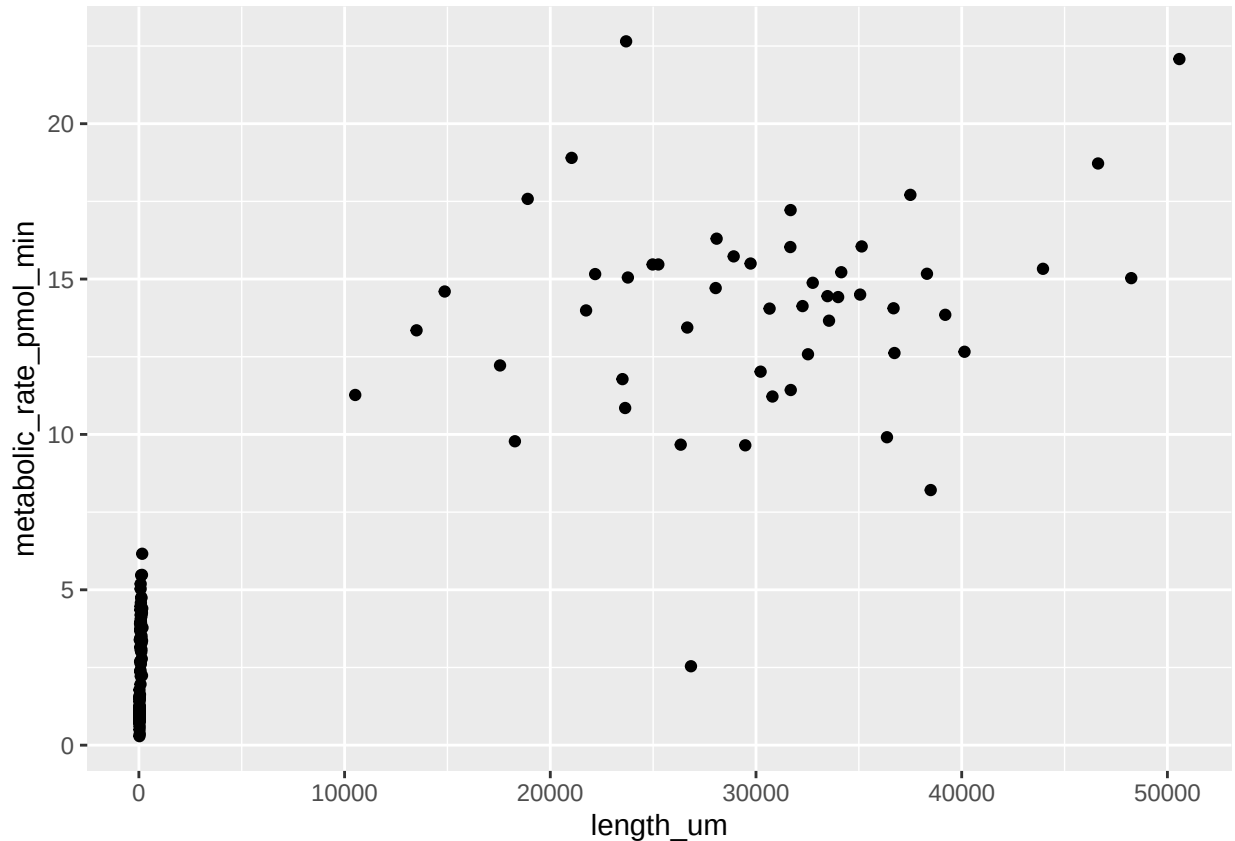


Something has popped up in our plots tab in the lower right. No data are plotted, but we have axes! There are no data because we haven't told R in what *layers* (*geoms*) we want to plot it in. All we have specified is what variables to put on the x and y axes.

## Step 2b: Plot a scatterplot

We are interested in the relationship between cell size and metabolic rate, both of which are continuous variables. This is a perfect application for a scatterplot using `geom_point()`.

```
ggplot(cell_data, aes(x = length_um, y = metabolic_rate_pmol_min)) + geom_point()
```

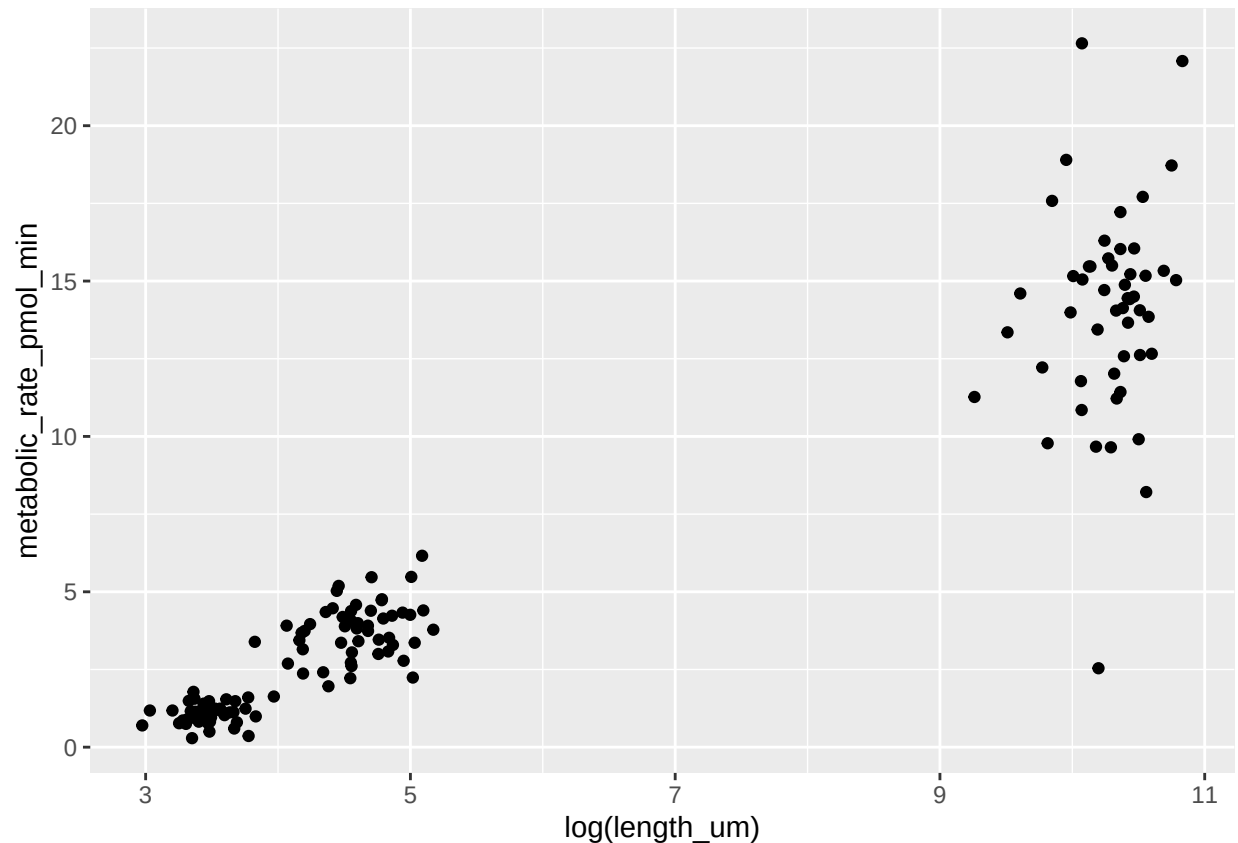


Something is not quite right here. Why do we have a stack of points lined up around zero length?

## Step 2c: Rescale variables

This is actually quite a common problem when we plot data like sizes, masses, distances, volumes, etc. because they can often vary by orders of magnitude. The issue is that the `length_um` variable needs to be *rescaled* to prevent the very large cells from overwhelming the plot. We are going to adjust the axis to a *natural log scale* by using the `log()` function. We could use `log()` within the `mutate()` function to add a column to the data with logged lengths, but `ggplot2` actually makes it possible to do that within the aesthetic.

```
ggplot(cell_data, aes(x = log(length_um), y = metabolic_rate_pmol_min)) + geom_point()
```



That's much better! We can now see the small cells ( $\sim 100$  micrometers) as well as the larger cells ( $\sim 8$  millimeters).

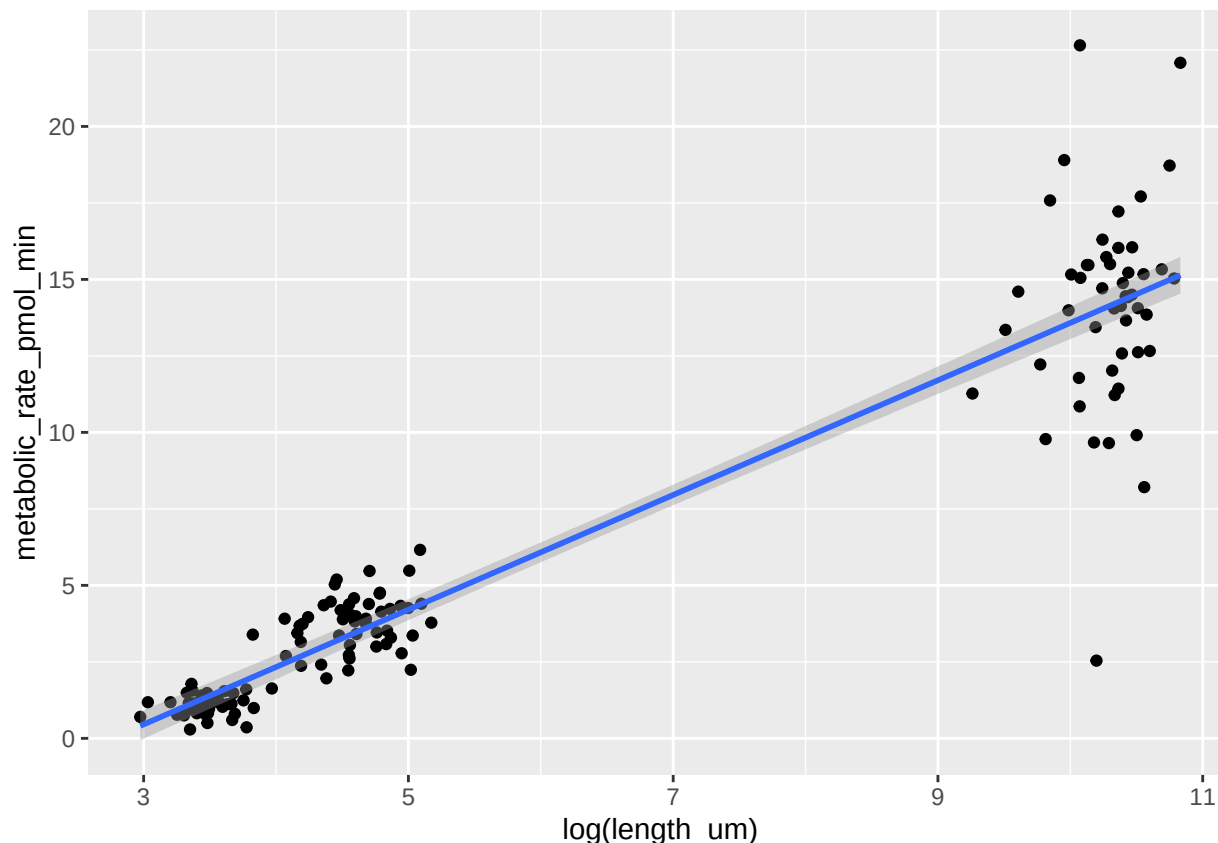
## Step 2d: Adding a trend line

Let's also add a "trend line" to the plot. The simplest trend line we can add is a linear trend line. We can tell the relationship here is not exactly linear (actually, it's log-linear), but it will do to visualize the general relationship.

Plot layers or `geoms()` get *added* together, so we can just *add* the trend line to our existing plot. The `geom()` for trend lines is called `geom_smooth()`, and it has several possible arguments. One important argument is *method*, which specifies the type of trend line (linear, nonlinear, etc.). The method we are going to use is "lm".

```
ggplot(cell_data, aes(x = log(length_um), y = metabolic_rate_pmol_min)) + geom_point() +  
  geom_smooth(method = "lm")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



## Step 2e: Rename the axis labels

The next thing we want to do is rename the axis labels. The labels from our tibble column names get the job done, but the lower-case letters and underscores aren't exactly publication-worthy. We can change the labels using the `labs()` function. Like in most cases where we want to add something to a plot, `labs()` just gets added on to the ggplot.

### Mini Challenge 1

Two arguments in `labs()` are *x* and *y*.

1. Copy and paste the ggplot we have been building
2. Change the axis labels to "Metabolic rate (pmol/min)" "and Log length (um)"

Use the code block below to code your answers!

```
#-----
```

## Step 2f: Colour points by group

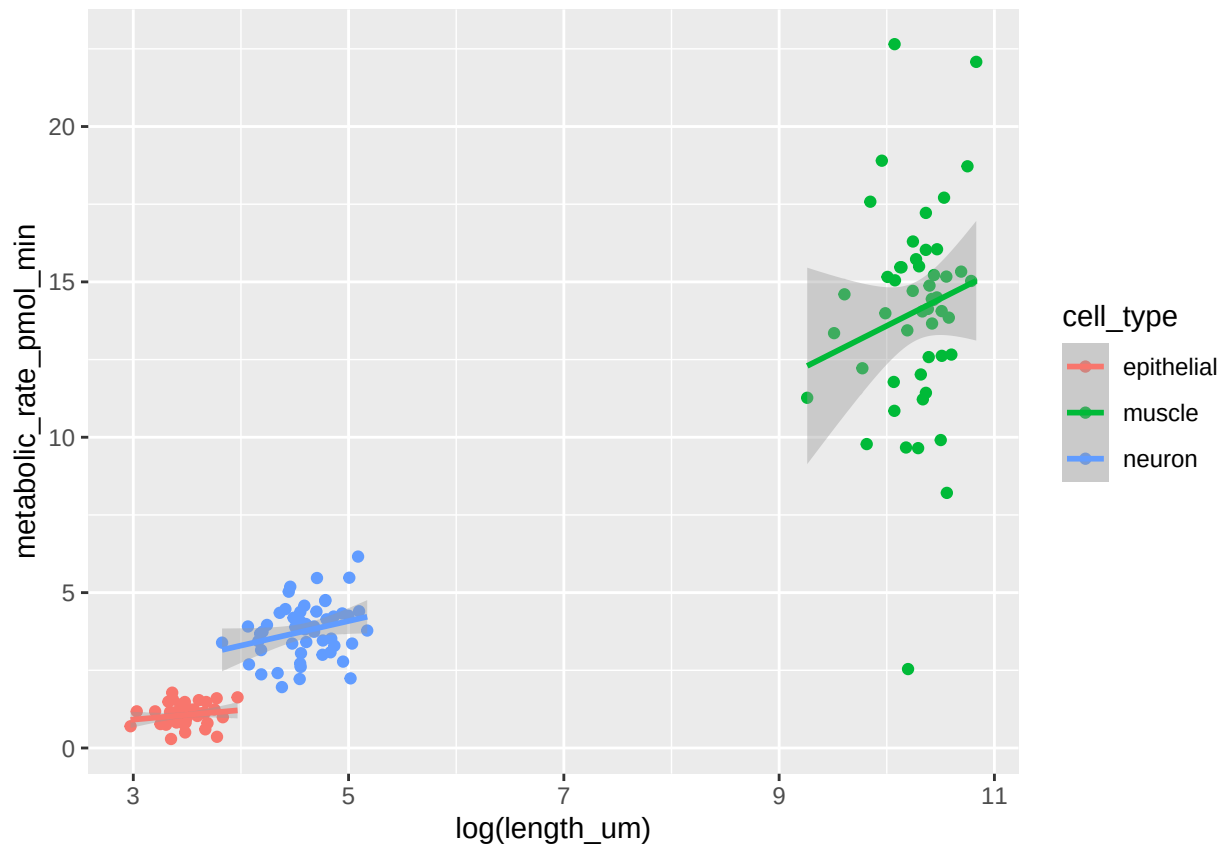
We will make one final adjustment to help us visualize the data. We might imagine the cluster of very large cells must be a different cell type than the smaller cells in the corner of the graph. However, we don't know which cluster belongs to which cell type. We can visualize the cell types if we *colour the graph by cell type*.

This operation is very similar to what `group_by()` does to our tibbles when we give it a column we want to group by—ggplot2 recognizes that the similar elements of a column now belong to a group.

Let's colour the points by `cell_type`. Colour is specified with the argument `colour` in the aesthetic (`aes()`), along with the x and y axes. The value of the argument is the name of the column we want to colour by.

```
ggplot(cell_data, aes(x = log(length_um), y = metabolic_rate_pmol_min, colour = cell_type)) +  
  geom_point() + geom_smooth(method = "lm")
```

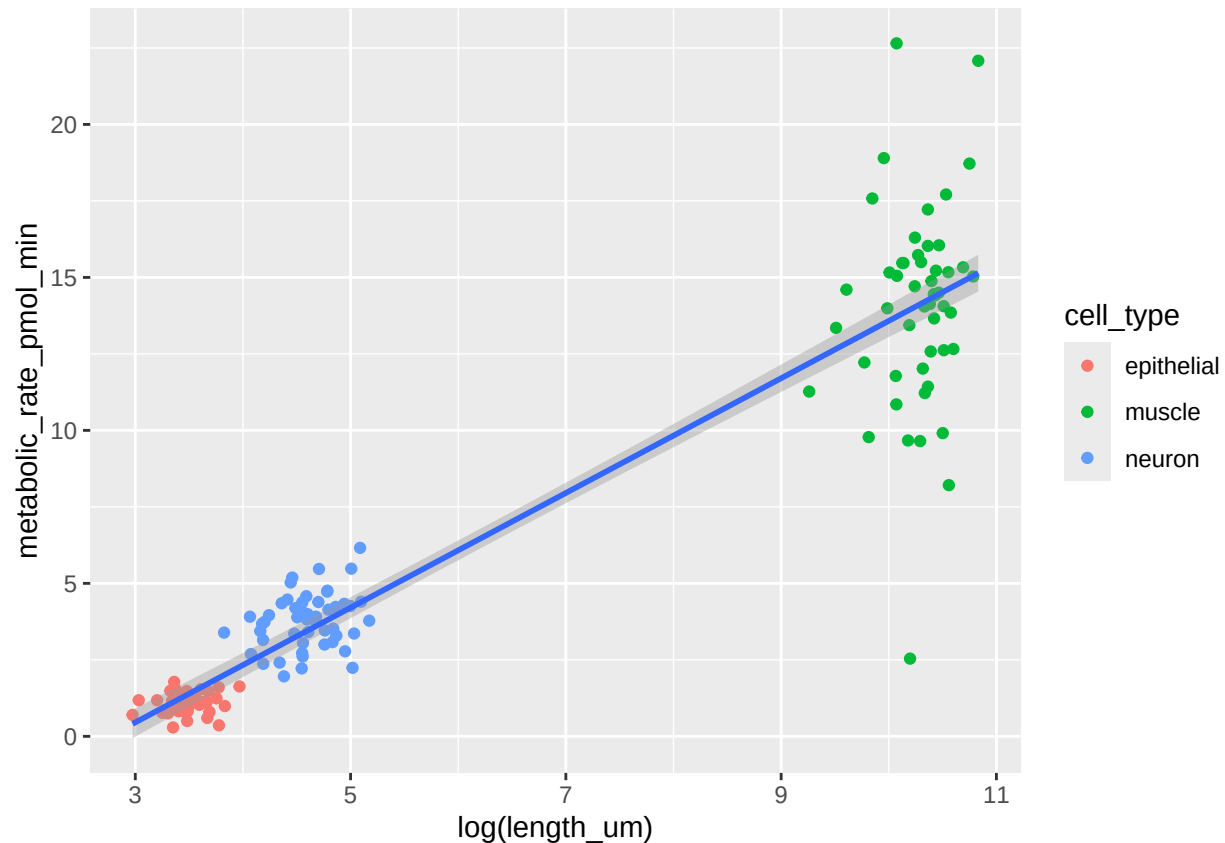
```
## `geom_smooth()` using formula = 'y ~ x'
```



We can now distinguish the cell type, but something strange happened! Our trend lines are now truncated and also coloured by cell type. This happened because we included colour in the *main* plot aesthetic, which tells ggplot2 to colour *all* of the layers that way. If we don't want to colour all of the layers, we can add a *second aesthetic function inside the geom() we want to colour*, and move the colour argument there.

```
ggplot(cell_data, aes(x = log(length_um), y = metabolic_rate_pmol_min)) + geom_point(aes(colour = cell_  
  geom_smooth(method = "lm")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Perfect! This is a pretty nice plot.

### 3 - Construct a boxplot

Next, we're going to construct a boxplot to compare circulating cortisol levels from our `cortisol_data`. Boxplots will allow us to visualize how the levels differ among stressors.

#### Step 3a: Basic boxplot

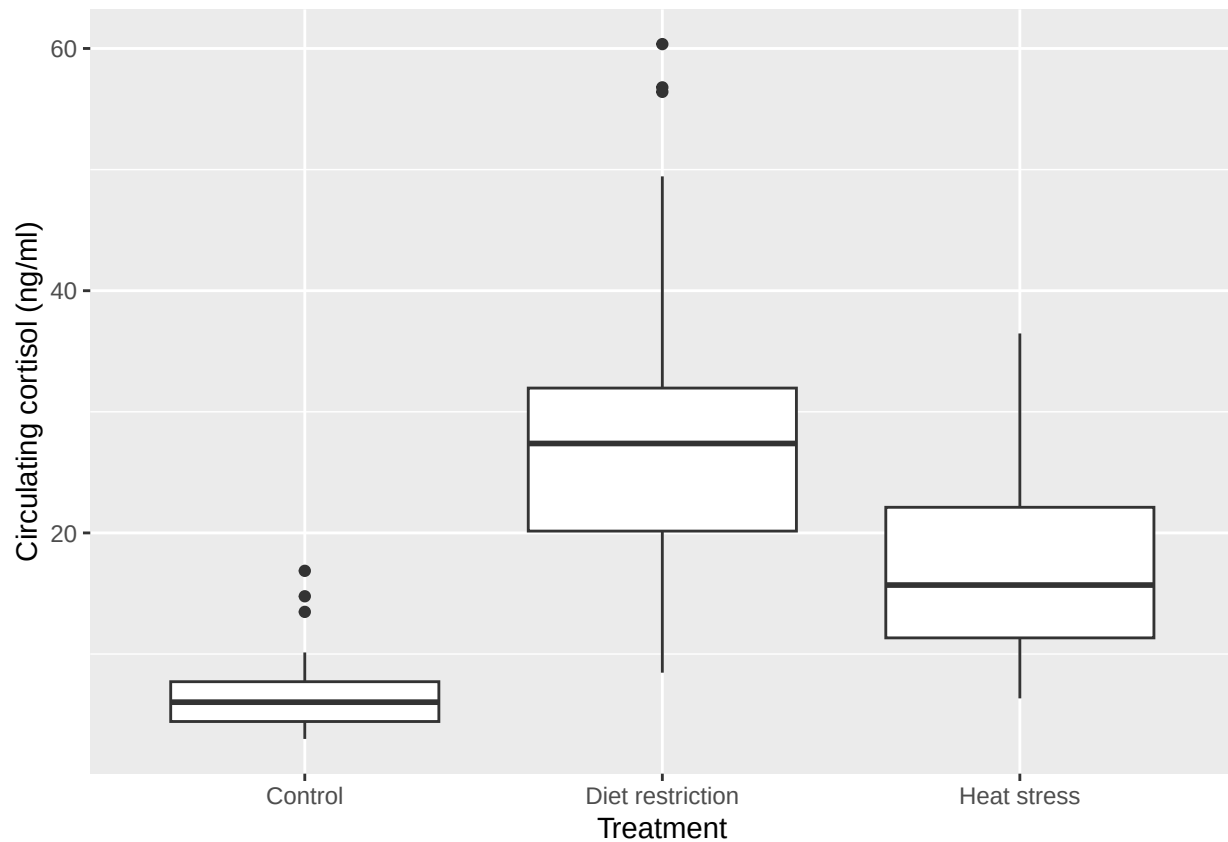
We follow the same process: Specify the data and global axes, which apply to all layers that follow, then add a `geom()`. The x (independent) variable is our now `treatment`, and the y (dependent variable) is `cortisol_ng_ml`. This time, however, we are going to do something different. If we assign the plot a name to make it an object in our environment, we can keep adding layers to the *object* rather than recreating the plot each time we want to add a new layer.

```
cort_plot <- ggplot(cortisol_data, aes(x = treatment, y = cortisol_ng_ml)) + geom_boxplot()
```

The plot now shows up as an object in our environment, but doesn't display in the viewing panel. To display it, we need to type the name of the plot.

While we're at it, we can add the labels just with the name of the plot object + `labs()`.

```
cort_plot + labs(x = "Treatment", y = "Circulating cortisol (ng/ml)")
```

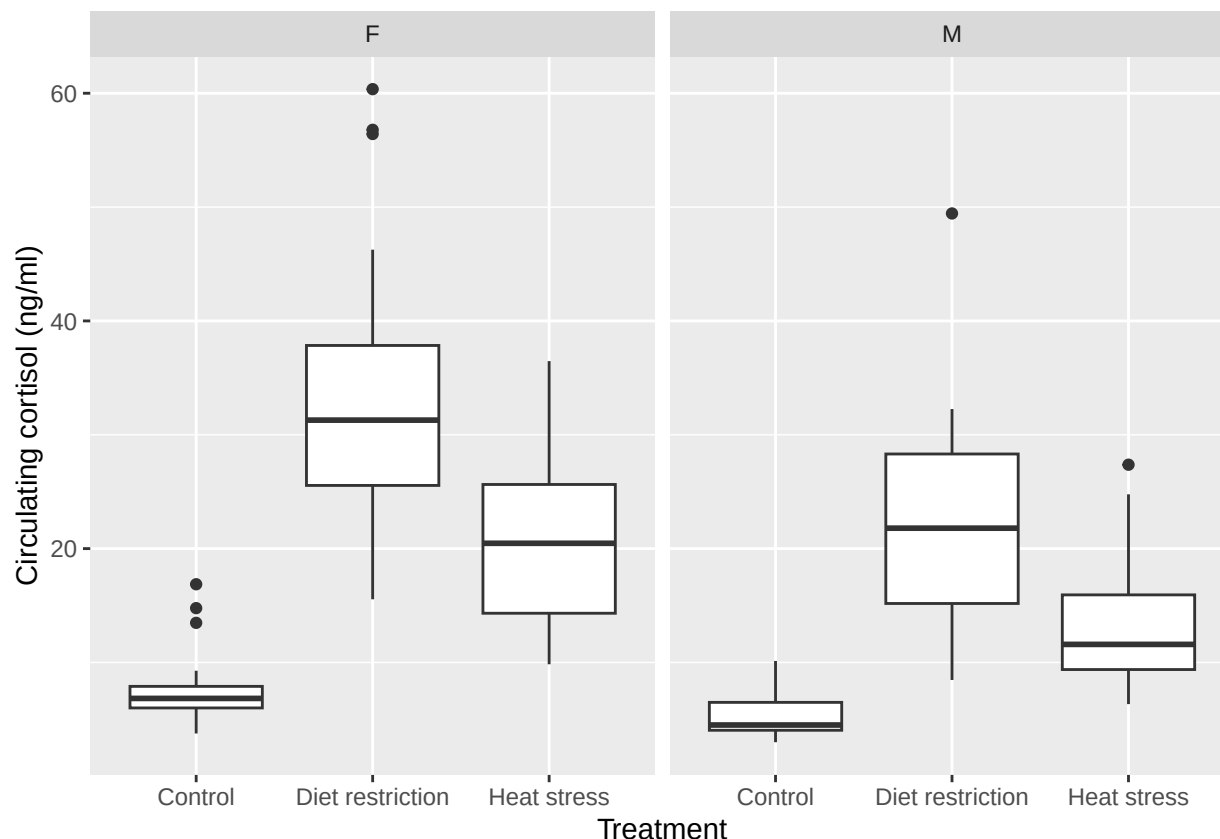


### Step 3b: Facetting

This looks fine, but didn't we also have two sexes in the data? What if there are differences in how males and females respond to the treatments? We might want to visualize that.

We already saw how we could distinguish groups with colour. Now, we will see how we can distinguish groups by plotting them in separate *panels*. The function for separating groups by panel is called `facet_wrap()`. The argument for `facet_wrap()` is the group, preceded by a tilde (~). Like other layers, it just gets added to the plot.

```
cort_plot + labs(x = "Treatment", y = "Circulating cortisol (ng/ml)") + facet_wrap(~sex)
```



Great! Now we can see sex differences in responses to the treatments. The “facet labels” appear above each panel.

However, the facet labels are currently just “F” and “M”. If we want the graph to be publication quality, we will want to change the labels to “Female” and “Male”. We also want to re-order the boxes so that the control appears first, followed by heat stress, and finally, diet restriction.

We have used the `factor()` function before, to turn data columns into `factor` data types, but we haven’t really encountered many applications where it is useful. We are going to see how `factor()` can both help us rename the values in our columns and re-order them in plots.

If you look at the `factor()` help files, you will see that it actually takes multiple arguments:

- **x**: The column or variable we want to turn into a `factor`
- **levels**: A vector of all the unique values in the column we want to `factor`, *which can be arranged in any order we want* (remember that R interprets `factors` as ordered integers)
- **labels**: A vector of any values, with the same number of elements as **levels**, which *allows us to rename the values* in the `factor` and map them on to the old **levels**

Here is an example of how this works:

```
# Create a character vector of unordered numbers
nums <- c("4", "6", "5")
# Create a vector of labels we want to rename with, in the correct order
num_names <- c("four", "five", "six")
# Factor and re-order the numbers in a new factor
num_fac <- factor(nums, levels = c("4", "5", "6"), labels = num_names)
```

```
# Print the new factor
num_fac
```

```
## [1] four six  five
## Levels: four five six
```

## Mini Challenge 2

1. Create a new **factor** column in `cortisol_data` called `"sex_f"`, relabelling "F" and "M" as "Female" and "Male"
2. Create another new **factor** column in `cortisol_data` called `"treatment_f"`, re-ordering the levels in the order: Control, Heat stress, Diet restriction
3. Assign the new data a name: `cortisol_data_factored`
4. Create a new `cort_plot` with the new data, starting with specifying the global axes (you can copy and paste the plot structure), and replacing the old variables with the new factors

Use the code block below to code your answers!

```
#-----
```

## Step 4 - Construct a line graph

Next, we are going to create a line graph. That is, *you* are going to create a line graph. You are going to use the `recovery_data` to visualize the patterns by which the heart returns to its resting rate after exercise. You are comparing patterns between people with Low, Moderate, and High fitness levels.

### Step 4a: Basic line graph

#### Mini Challenge 3

The `geom()` function to create a line graph is `geom_line()`.

1. Create a basic graph with `heart_rate_bpm` as the response variable and `time_min` as the explanatory variable
2. Add the layer for the line `geom()`, colouring by `fitness_level`
3. Change the axis labels to "Time (minutes)" "Heart rate (beats per minute)"
4. Assign the plot the name `"recovery_plot"`
5. Visualize the plot

Use the code block below to code your answers!

```
#-----
```

### Step 4b: Customize your colours

This looks pretty good! However, these colours are getting a bit boring, don't you think? Wouldn't it be nicer if you could customize your colours?

Well, you can! The colour aesthetic belongs to the realm of ggplot *scales*. Scales control how data get mapped to the visual elements of your plot. Data elements you can scale include:

- axis breaks
- legends
- shapes
- sizes
- line types
- *and* colours

We are going to use a function/layer called `scale_colour_manual()`. The most important argument in the function is `values`, which requires a vector of colours. The syntax is, “values, equals, vector of colours”. The colour vector must:

- be a character vector, each element specifying a colour
- include the same number of colour elements as groups you have coloured in your plot

You already saw that R associates some names with specific colours. For example:

```
"orange"
```

```
## [1] "orange"
```

```
"red"
```

```
## [1] "red"
```

```
"blue"
```

```
## [1] "blue"
```

There are also some interesting variations:

```
"darkgoldenrod1"
```

```
## [1] "darkgoldenrod1"
```

```
"deeppink"
```

```
## [1] "deeppink"
```

```
"cornflowerblue"
```

```
## [1] "cornflowerblue"
```

R also recognizes something called *hex codes*. These are alphanumeric strings that represent colours. Unlike the R names for colours, there are countless variations of hex codes that make virtually any colour possible. Hex codes are entered as characters, just like colour names, with hashtags at the beginning of the alphanumeric string:

```
# Black  
"#000000"
```

```
## [1] "#000000"
```

```
# Red  
"#FF0000"
```

```
## [1] "#FF0000"
```

```
# Pink  
"#FFDDDD"
```

```
## [1] "#FFDDDD"
```

### Mini Challenge 4

The website [color-hex.com](http://color-hex.com) provides the hex codes for a huge variety of colours.

1. Create a character vector called *fitness\_cols*, containing the required number of hex codes for the fitness levels in *recovery\_data*, using any colours you like from [color-hex.com](http://color-hex.com)
2. Use the `scale_colour_manual()` function to recolour the fitness level lines in *recovery\_plot*

Use the code block below to code your answers!

```
#-----
```

## Step 5 - Construct a bar chart

The final plot we are going to construct is a bar chart. We are going to use the fish data to visualize the proportions of fish that survived at each stocking density.

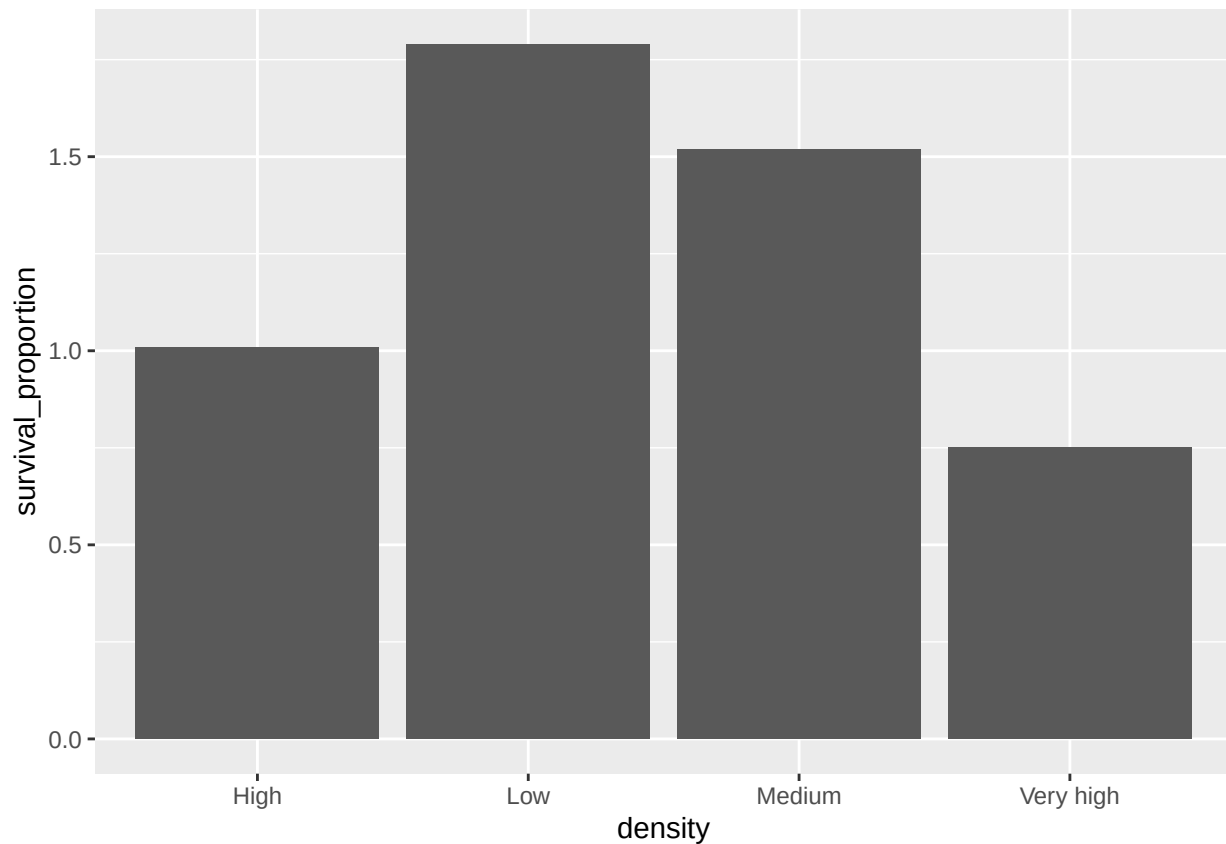
### Step 5a: Basic bar chart

The `geom()` function for bar charts is `geom_bar()`. We want to compare `survival_proportions` (y) between `density` treatments (x).

```
ggplot(fish_data, aes(x = density, y = survival_proportion)) + geom_bar()
```

It didn't work! Unlike the other `geom()` functions, `geom_bar()` has a necessary argument. If you look at the help files for this function, you will see the argument *stat*. By default, the value for this argument is "count". What this means is that the values in the column are counted and the sum is used as the response variable. However, the counts were already calculated: `n_survived` is the number of surviving fish, and `survival_proportion` is the proportion of surviving fish out of those stocked. This means we can use the values in the column *directly*. To do this, we need to specify `stat = "identity"` in the `geom_bar()` function.

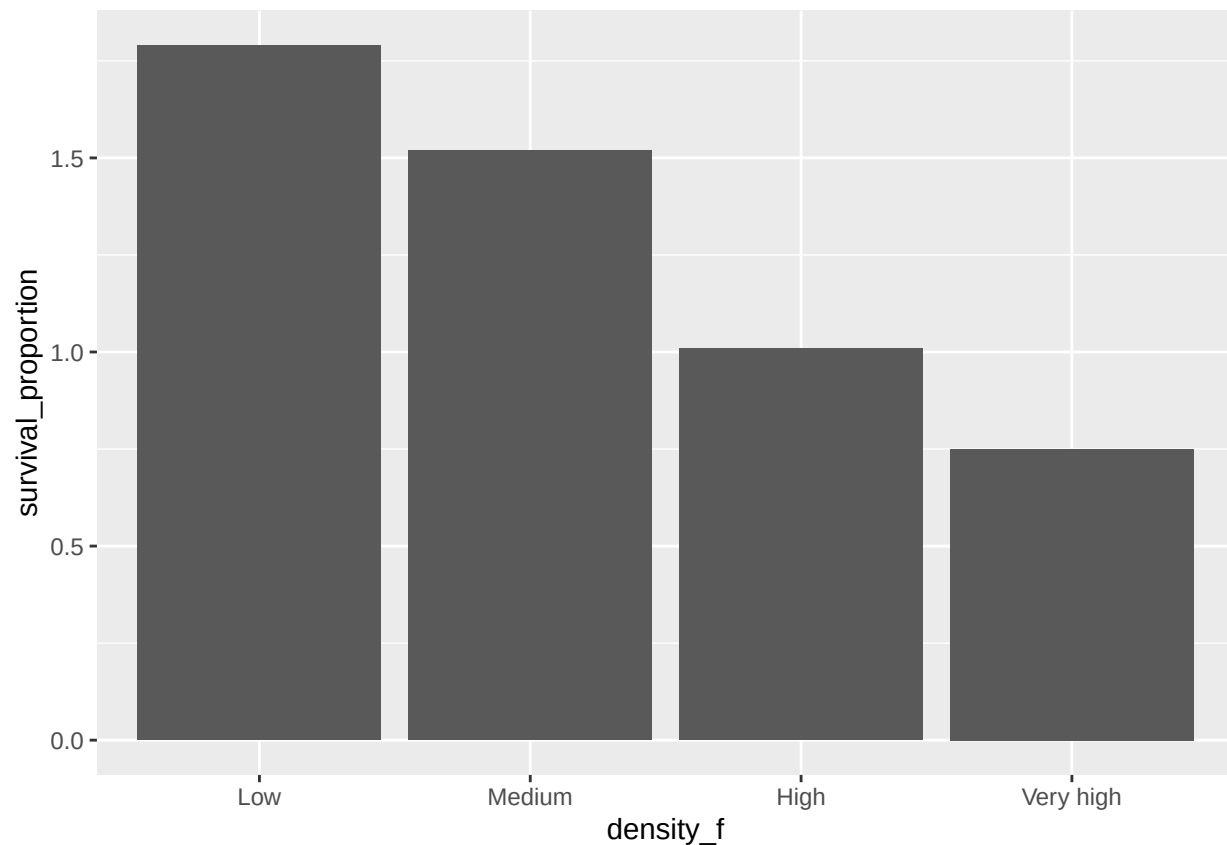
```
ggplot(fish_data, aes(x = density, y = survival_proportion)) + geom_bar(stat = "identity")
```



## Step 5b: Factor the treatments

That's better. However, you can immediately see that the density treatments are out of order. You know they should go from Low to Very high, but they are currently ordered: High, Low, Medium, Very high. Again, we can fix this by adding a new column with the treatments as a factor.

```
# Add a new factor for density treatment
fish_data_factored <- fish_data %>%
  mutate(density_f = factor(density, levels = c("Low", "Medium", "High", "Very high")))
ggplot(fish_data_factored, aes(x = density_f, y = survival_proportion)) + geom_bar(stat = "identity")
```

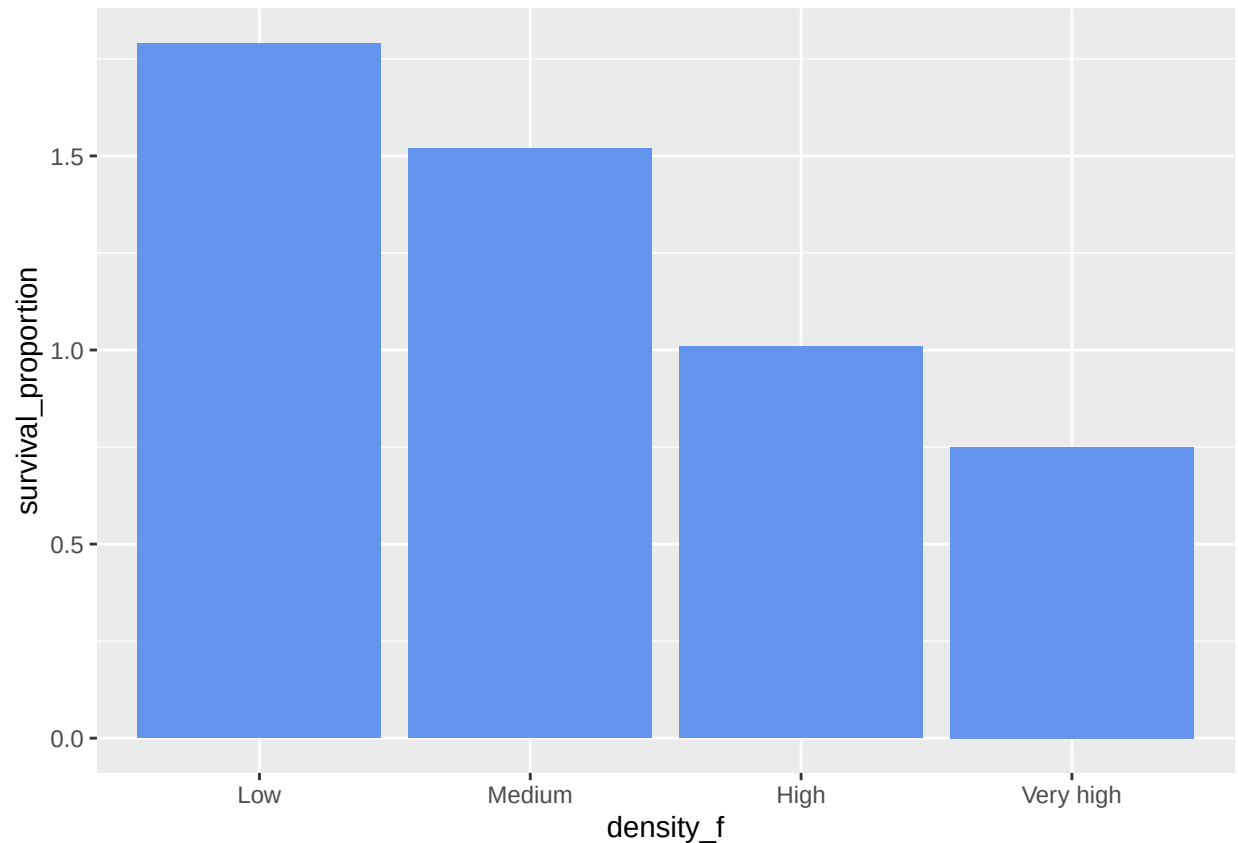


### Step 5c: Fill the bars by a variable

Let's add some colour to the plot. Earlier, we saw that we can specify a colour in the aesthetic of the plot or the aesthetic of a `geom()` to distinguish groups in the data. Alternatively, we can recolour *all* the data with the same colour by specifying the colour outside of the aesthetic.

Points, lines, and other single dimensional `geoms()` are recoloured with the `colour` argument. However, `geoms()` like boxplots and bars are *2-dimensional*. To recolour them, we need a new argument: `fill`.

```
ggplot(fish_data_factored, aes(x = density_f, y = survival_proportion)) + geom_bar(fill = "cornflowerblue",  
  stat = "identity")
```

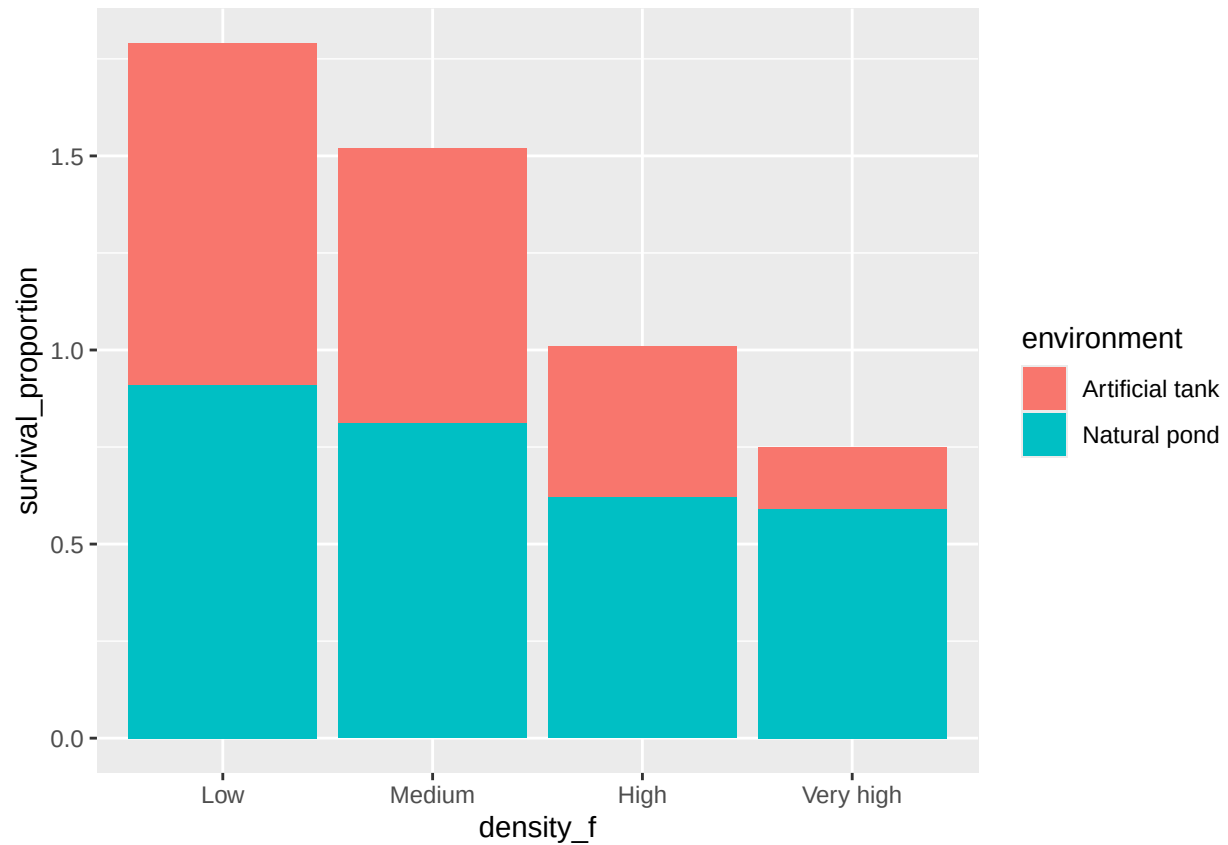


That's a nice plot! But something is off—*why do some of the survival proportions add to values  $> 1$ ?*

The problem is that we have two *environments* in our data: “Artificial tanks” and “Natural ponds”. When we plot the data with a bar chart, ggplot2 stacks both environments on top of each other, resulting in survival proportions  $> 1$ .

To fix the issue, *we can just go back to colouring within the aesthetic*. If we colour by environment, we should end up with two bars for each density treatment representing the different environments.

```
ggplot(fish_data_factored, aes(x = density_f, y = survival_proportion)) + geom_bar(aes(fill = environment, stat = "identity"))
```



By default, the bars are “stacked”. We can specify separate bars for each environment using the `position` argument in `geom_bar`, which we will set to “*dodge*” (the default is “*stack*”).

```
ggplot(fish_data_factored, aes(x = density_f, y = survival_proportion)) + geom_bar(aes(fill = environment, stat = "identity", position = "dodge"))
```

